

Politechnika Gdańska

Sprawozdanie z projektu

Przedmiot: Basics of Time Series

**Analiza szeregu czasowego:
Individual Household Electric Power Consumption**

Wykonał: Juliusz Radziszewski

Nr indeksu: s193504

Data: 16.05.2026 r.

Część 1. Wybór szeregu czasowego i jego opis

Charakterystyka analizowanego zagadnienia

Analizowanym zagadnieniem jest zapotrzebowanie na energię elektryczną w pojedynczym gospodarstwie domowym. Zrozumienie dynamiki takich szeregów jest kluczowe dla prognozowania obciążeń sieci energetycznych (tzw. smart grids), identyfikacji wzorców sezonowych oraz optymalizacji zużycia prądu przez odbiorców końcowych.

Charakterystyka szeregu, meta-dane i źródło

- **Źródło:** UCI Machine Learning Repository (udostępnione pierwotnie przez Georges'a Hebraila z EDF - Électricité de France).
- **Wielkość:** Oryginalny zbiór zawiera 2 075 259 obserwacji.
- **Odstęp czasowy (rozdzielczość):** 1 minuta (na potrzeby niniejszej analizy dane zostały zagregowane do rozdzielczości 1 dnia).
- **Okres:** Dokładny zakres analizowanych danych po agregacji obejmuje okres od **16 grudnia 2006 r. do 26 listopada 2010 r.** (prawie 4 lata).
- **Analizowana zmienna:** `Global_active_power` – uśredniona w czasie globalna moc czynna gospodarstwa domowego.
- **Jednostka:** Kilowaty (kW).
- **Jakość danych:** Występują braki danych, które w surowym pliku oznaczone były znakiem zapytania (?). Stanowiły one około 1.25% początkowego zbioru i zostały automatycznie obsłużone podczas agregacji do średnich dziennych.

Wykorzystane polecenia R (part1.R)

Poniżej przedstawiono zaktualizowany skrypt użyty do wczytania zbioru, wyznaczenia dokładnego horyzontu czasowego, agregacji oraz zapisu poprawionego wykresu liniowego z wyrównaną osią czasu.

```
# 1. Deklaracja "importów" - niezbędne do działania funkcji
library(dplyr)
library(lubridate)
library(ggplot2)

# 2. Wczytanie surowego zbioru danych z pliku (plik musi być w folderze roboczym R)
cat("Wczytywanie danych... To może potrwać kilka-kilkanaście sekund.\n")
df <- read.csv("household_power_consumption.txt", sep=";", na.strings="?",
stringsAsFactors=FALSE)

# 3. Przetwarzanie formatu daty na rozpoznawalny przez R
df$Date <- dmy(df$Date)

# 4. Agregacja do poziomu dziennego (teraz operator %>% zadziała bez problemu)
df_daily <- df %>%
  group_by(Date) %>%
```

```

    summarise(Daily_Global_Active_Power = mean(Global_active_power, na.rm =
TRUE))

# Odrzucamy dni z pustymi danymi
df_daily <- na.omit(df_daily)

# Sprawdzenie i wypisanie dokładnych dat początkowych i końcowych
data_początkowa <- min(df_daily$Date)
data_koncowa <- max(df_daily$Date)
cat("Początek danych:", format(data_początkowa, "%Y-%m-%d"), "\n")
cat("Koniec danych:", format(data_koncowa, "%Y-%m-%d"), "\n")

# 5. Generowanie wykresu szeregu czasowego z wymuszoną osią X
wykres <- ggplot(df_daily, aes(x = Date, y = Daily_Global_Active_Power)) +
  geom_line(color = "darkblue", linewidth = 0.5) +
  scale_x_date(date_breaks = "1 year", date_labels = "%Y") + # Wymuszenie
równych lat na osi
  labs(title = sprintf("Średnie dzienne zużycie energii elektrycznej (%s do
%s)",
                                format(data_początkowa, "%Y"), format(data_koncowa,
"%Y")),
        subtitle = "Zagregowane dane z 1-minutowych pomiarów w gospodarstwie
domowym",
        x = "Data",
        y = "Średnia moc czynna [kW]",
        caption = "Źródło: UCI Machine Learning Repository") +
  theme_minimal()

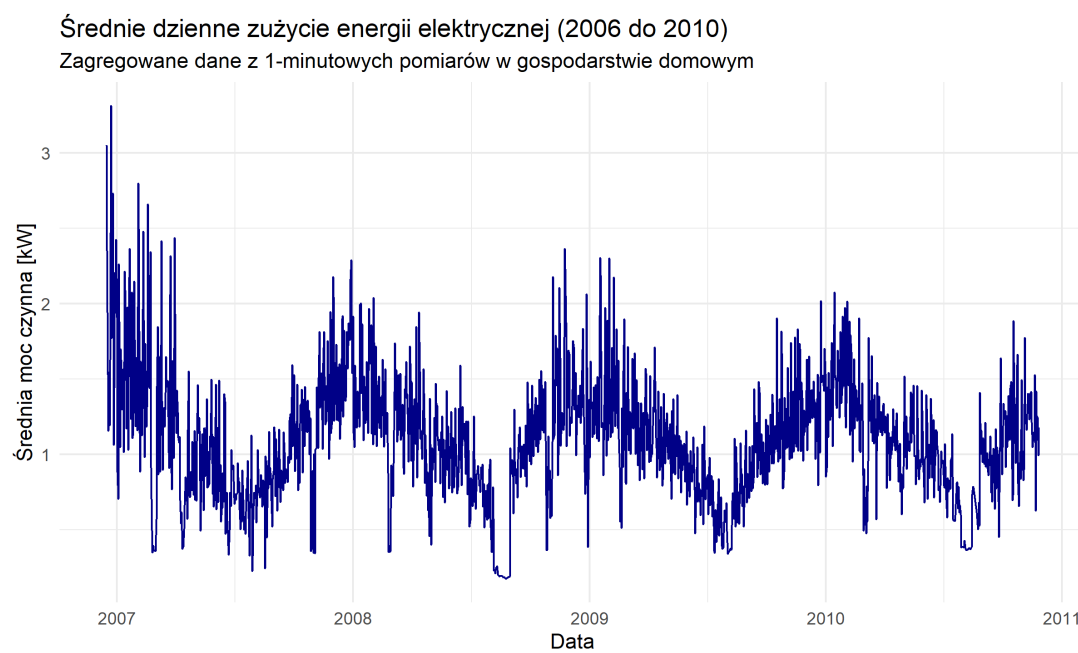
# Wyświetlenie wykresu w środowisku
print(wykres)

# 6. Automatyczny zapis poprawionego wykresu do pliku
ggsave("part1-photo.png", plot = wykres, width = 8, height = 5, dpi = 300)
cat("Zapisano wykres do pliku part1-photo.png\n")

```

Wykres szeregu czasowego

Poniższa wizualizacja prezentuje uśrednione dzienne zużycie mocy czynnej.



Rysunek 1: Średnie dzienne zużycie energii elektrycznej w okresie od grudnia 2006 do listopada 2010 r.

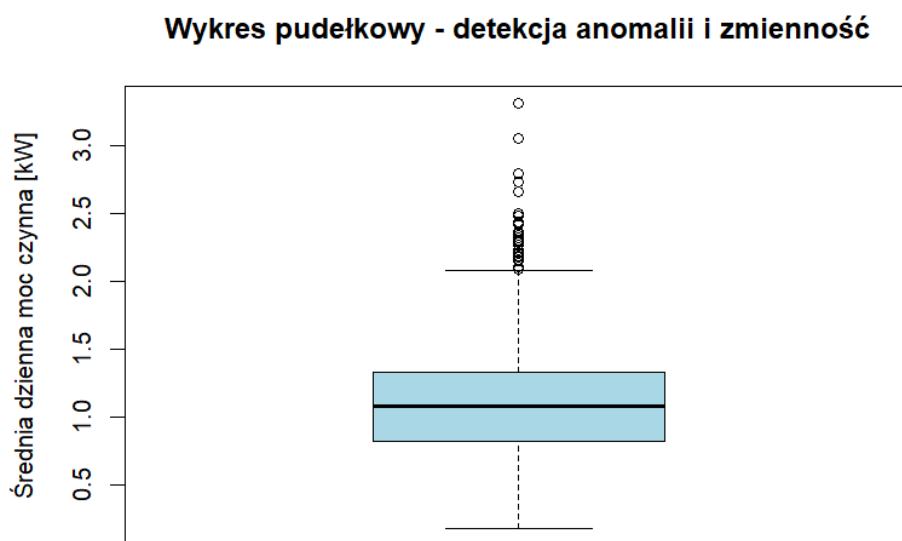
Część 2. Analiza wstępna wybranego szeregu czasowego

Konwersja na obiekt szeregu czasowego

Po wczytaniu danych i ich zagregowaniu do rozdzielczości dziennej, za pomocą funkcji `is.ts()` sprawdzono, czy wektor wartości jest rozpoznawany przez środowisko R jako szereg czasowy. Wynik był negatywny (`FALSE`). W związku z tym użyto funkcji `ts()` z parametrem `frequency = 365`, aby poprawnie przekonwertować dane na dedykowany obiekt szeregu czasowego (`ts_power`).

Anomalie, wartości odstające i brakujące

W surowym zbiorze 1-minutowym występowały braki danych (ok. 1.25%), jednak zostały zebrane i oczyszczone podczas dziennej agregacji (funkcja `na.omit()`). Aby wykryć anomalie i ocenić rozkład danych, wygenerowano wykres pudełkowy (Boxplot). Ukazuje on obecność wartości odstających – widocznych jako punkty powyżej górnego wąsa rozkładu. Reprezentują one specyficzne dni o wyjątkowo wysokim zużyciu energii, co w kontekście gospodarstwa domowego może być spowodowane np. szczytem sezonu grzewczego lub intensywnym wykorzystaniem urządzeń AGD.



Rysunek 2: Wykres pudełkowy reprezentujący zmienność oraz wartości odstające szeregu.

Składowa stała i zmienność

Składowa stała: Jako miarę składowej stałej wybrano średnią arytmetyczną, ponieważ oddaje bazowy poziom obciążenia sieci. Wynosi ona dokładnie 1.09 kW. **Zmienność:** Do pomiaru zmienności wybrano odchylenie standardowe (0.42 kW) oraz współczynnik zmienności (CV), który wyniósł 38.50%. Współczynnik zmienności

wybrano ze względu na jego relatywny charakter – pozwala on na obiektywną ocenę rozrzutu danych niezależnie od skali zjawiska.

Analiza dynamiki

Dynamikę szeregu zbadano za pomocą średniego tempa zmian opartych na indeksach łańcuchowych (zmiana z dnia na dzień). Średnia wartość indeksu wyniosła **105.74%**. Wybór wskaźników łańcuchowych podyktowany jest dużą zmiennością krótkoterminową (dzienną) – zużycie prądu w gospodarstwie domowym zależy silnie od cyklu tygodniowego (dni robocze vs weekendy).

Charakterystyka modelu (addytywny vs multiplikatywny)

Wizualna analiza szeregu wskazuje, że wykazuje on cechy modelu **addytywnego**. Wahania sezonowe (amplituda różnic pomiędzy podwyższonym zużyciem zimą a obniżonym latem) utrzymują się na w miarę stałym poziomie. W przypadku modelu multiplikatywnego amplituda ulegałaby proporcjonalnemu poszerzeniu wraz z ogólnym wzrostem lub spadkiem trendu, co w tym szeregu nie występuje.

Badanie stacjonarności

Stacjonarność szeregu sprawdzono za pomocą rozszerzonego testu Dickeya-Fullera (Augmented Dickey-Fuller Test). Otrzymano wartość p-value na poziomie **0.01** (przy statystyce testowej Dickey-Fuller = **-5.1735**). Ponieważ wartość p-value jest mniejsza od standardowego poziomu istotności $\alpha = 0.05$, odrzucamy hipotezę zerową o obecności pierwiastka jednostkowego. Oznacza to, że analizowany szereg czasowy **jest stacjonarny**.

Wykorzystane polecenia R (part2.R)

```
library(dplyr)
library(lubridate)
library(tseries)

# 1. Konwersja na szereg czasowy (zakładamy, że ramka df_daily z Części 1
# jest w pamięci)
cat("Czy df_daily$Daily_Global_Active_Power to ts?:",
    is.ts(df_daily$Daily_Global_Active_Power), "\n")
ts_power <- ts(df_daily$Daily_Global_Active_Power, frequency = 365)
cat("Czy ts_power to ts?:", is.ts(ts_power), "\n\n")

# 2. Zapis wykresu pudełkowego (wskaźnik zmienności / anomalie) do pliku
png("part2-boxplot.png", width=800, height=600, res=120)
boxplot(ts_power, main="Wykres pudełkowy - detekcja anomalii i zmienność",
        ylab="Średnia dzienna moc czynna [kW]", col="lightblue")
dev.off()

# 3. Składowa stała i zmienność
srednia <- mean(ts_power)
odchylenie <- sd(ts_power)
wsp_zmiennosci <- (odchylenie / srednia) * 100
```

```

cat("--- WSKAŹNIKI STATYSTYCZNE ---\n")
cat("Składowa stała (średnia):", round(srednia, 2), "kW\n")
cat("Zmienność (Odch. std.):", round(odchylenie, 2), "kW\n")
cat("Współczynnik zmienności (CV):", round(wsp_zmienności, 2), "%\n\n")

# 4. Dynamika (indeksy łańcuchowe)
dynamika <- (ts_power[-1] / ts_power[-length(ts_power)]) * 100
srednia_dynamika <- mean(dynamika)
cat("--- DYNAMIKA ---\n")
cat("Średnie tempo zmian (dzień do dnia):", round(srednia_dynamika, 2), "%\n\n")

# 5. Stacjonarność (Rozszerzony test Dickeya-Fullera)
cat("--- TEST STACJONARNOŚCI (ADF) ---\n")
print(adf.test(ts_power, alternative = "stationary"))

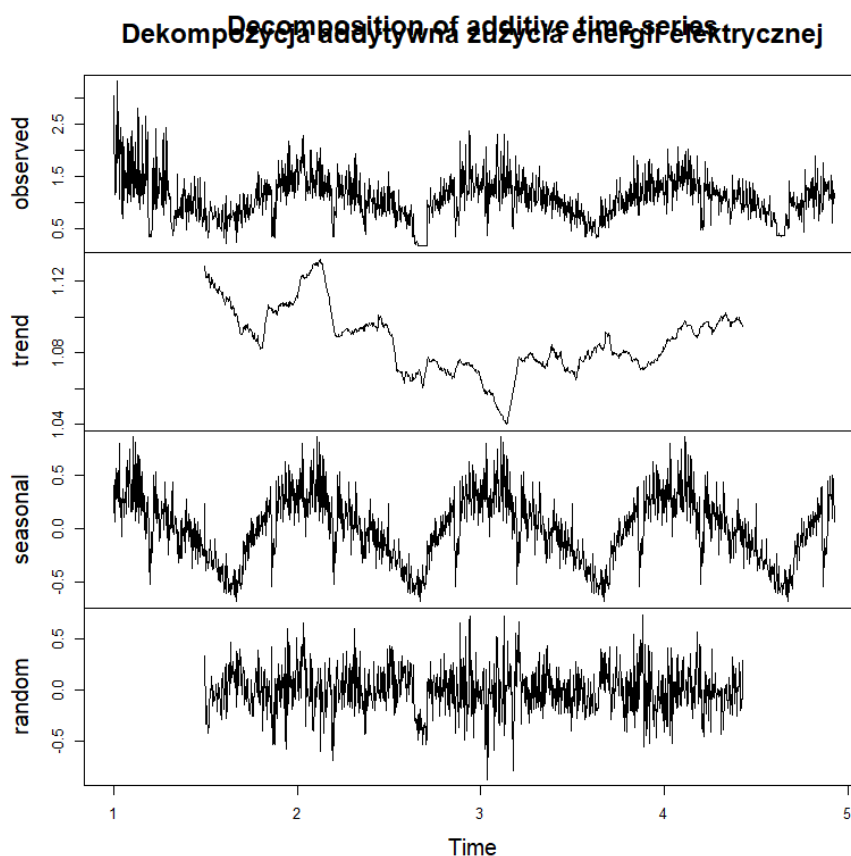
```

Część 3. Dekompozycja szeregu czasowego

Krytyczna ocena klasycznej dekompozycji

Pierwotnie do dekompozycji szeregu wykorzystano automatyczną funkcję `decompose()`. Narzędzie to bazuje na klasycznej średniej ruchomej. Ponieważ zdefiniowano roczną częstotliwość danych (365 dni), algorytm filtrujący do obliczenia wartości trendu potrzebował okna obejmującego pół roku w przód i pół roku w tył.

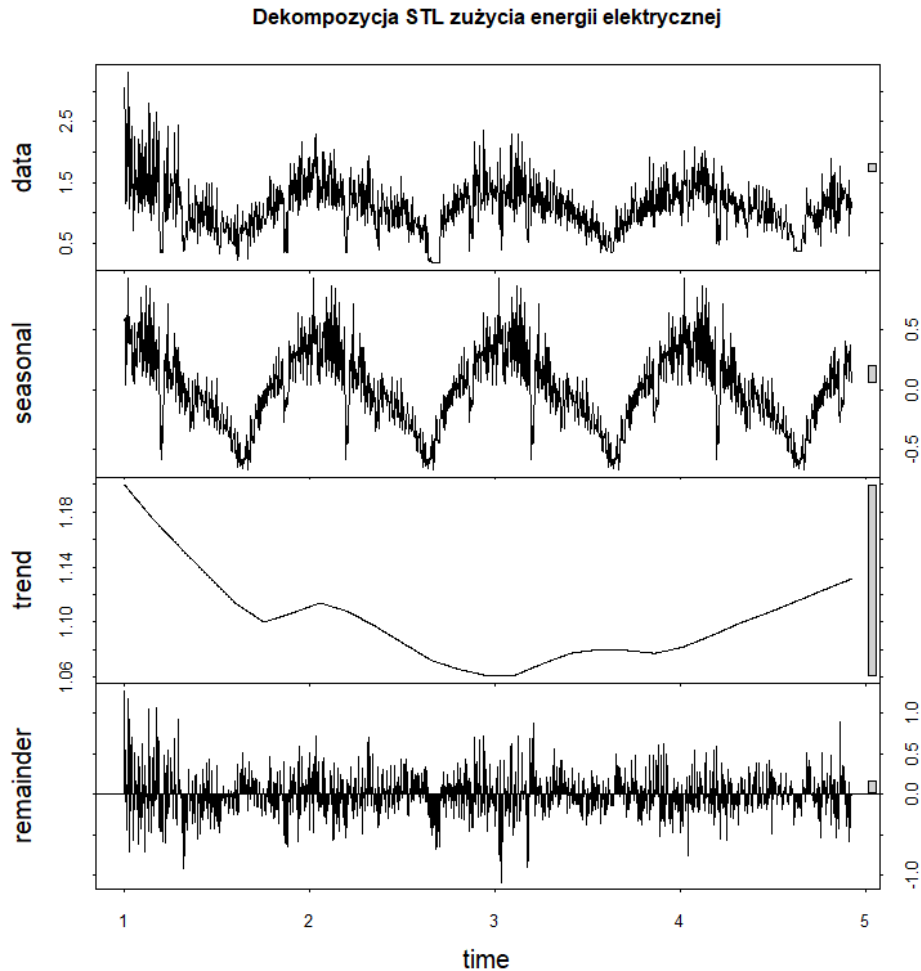
Skutkiem ubocznym tej metody była **utrata danych na krawędziach szeregu** – funkcja nie była w stanie wyznaczyć składowej trendu oraz szumu dla pierwszych i ostatnich 182 dni badanego okresu. Objawiło się to pustymi obszarami (lukami) na brzegach wykresu `part3-dekompozycja.png`. Zgodnie z zaleceniami, ta klasyczna metoda została uznana za niewystarczającą dla pełnego zbioru.



Rysunek 3: Klasyczna dekompozycja addytywna. Widoczne wyraźne luki i odcięte krawędzie składowej trendu i szumu (random).

Alternatywne rozwiązanie: Algorytm STL

Aby rozwiązać problem niedoskonałości klasycznej dekompozycji ruchomej, zastosowano nowoczesny algorytm **STL** (Seasonal and Trend decomposition using Loess). Wykorzystuje on lokalne regresje wygładzające (Loess), dzięki czemu efektywnie estymuje składowe na samych brzegach dziedziny czasu, nie powodując utraty cennych informacji z pierwszego i ostatniego roku obserwacji.



Rysunek 4: Zaawansowana dekompozycja STL. Wszystkie składowe zostały wyznaczone kompletnie na pełnym przedziale czasowym.

Interpretacja wyników dekompozycji STL

Zastosowanie algorytmu STL pozwoliło na pomyślne i precyzyjne wyodrębnienie wszystkich trzech składowych w sposób nieprzerwany (od początku do końca szeregu):

1. **Trend:** Dzięki wygładzaniu metodą Loess, linia trendu jest płynna i czytelna. Wykres wyraźnie pokazuje, że bazowe zużycie energii było najwyższe na początku badanego okresu, po czym stopniowo malało, osiągając swoje minimum (ok. 1.06 kW) na początku 3. roku obserwacji. Następnie trend ulega odwróceniu i zaczyna miarowo rosnąć. Warto jednak zauważyć (patrzac na szary pasek skali po prawej stronie panelu), że całkowita zmienność trendu jest relatywnie niewielka w porównaniu do wahań sezonowych.
2. **Seasonal (Sezonowość):** Algorytm idealnie wyizolował powtarzalny, roczny cykl o stałej amplitudzie. Wzorec ten potwierdza piki obciążenia sieci energetycznej w chłodniejszych miesiącach (wzmoczone ogrzewanie, krótszy czas operowania światła słonecznego) oraz głębokie dolki w okresie letnim.
3. **Remainder (Reszty/Szum):** Składnik losowy prawidłowo oscyluje wokół osi zera, co potwierdza dobre dopasowanie modelu addytywnego. Na wykresie reszt

szczególnie odznaczają się ostre „szpilki” skierowane w dół (wartości spadające lokalnie nawet do -1.0). Reprezentują one ewidentne anomalie – dni gwałtownego i niespodziewanego spadku zużycia energii, co w gospodarstwie domowym odpowiada najpewniej dłuższym nieobecnościom domowników (np. wyjazdom wakacyjnym).

Wykorzystane polecenia R (part3.R)

```
# 1. Automatyczna dekompozycja szeregu (model addytywny)
dekompozycja <- decompose(ts_power, type = "additive")

# 2. Zapisanie wykresu klasycznej dekompozycji do pliku
png("part3-dekompozycja.png", width=800, height=800, res=120)
plot(dekompozycja)
title(main="Dekompozycja addytywna zużycia energii elektrycznej", line=2)
dev.off()

# 3. Zastosowanie algorytmu STL (nie ucina brzegów szeregu)
dekompozycja_stl <- stl(ts_power, s.window = "periodic")

# 4. Zapisanie wykresu nowej dekompozycji STL do osobnego pliku
png("part3-stl.png", width=800, height=800, res=120)
plot(dekompozycja_stl, main="Dekompozycja STL zużycia energii
elektrycznej")
dev.off()
cat("Zapisano wykresy dekompozycji do plików part3-dekompozycja.png oraz
part3-stl.png\n")
```

Część 4. Wybrana analiza zaawansowana

Cel badania i metodyka z podziałem Train-Test

Cel: Zbudowanie optymalnego modelu predykcyjnego w celu prognozy średniego dziennego zapotrzebowania na energię elektryczną na horyzont 30 dni.

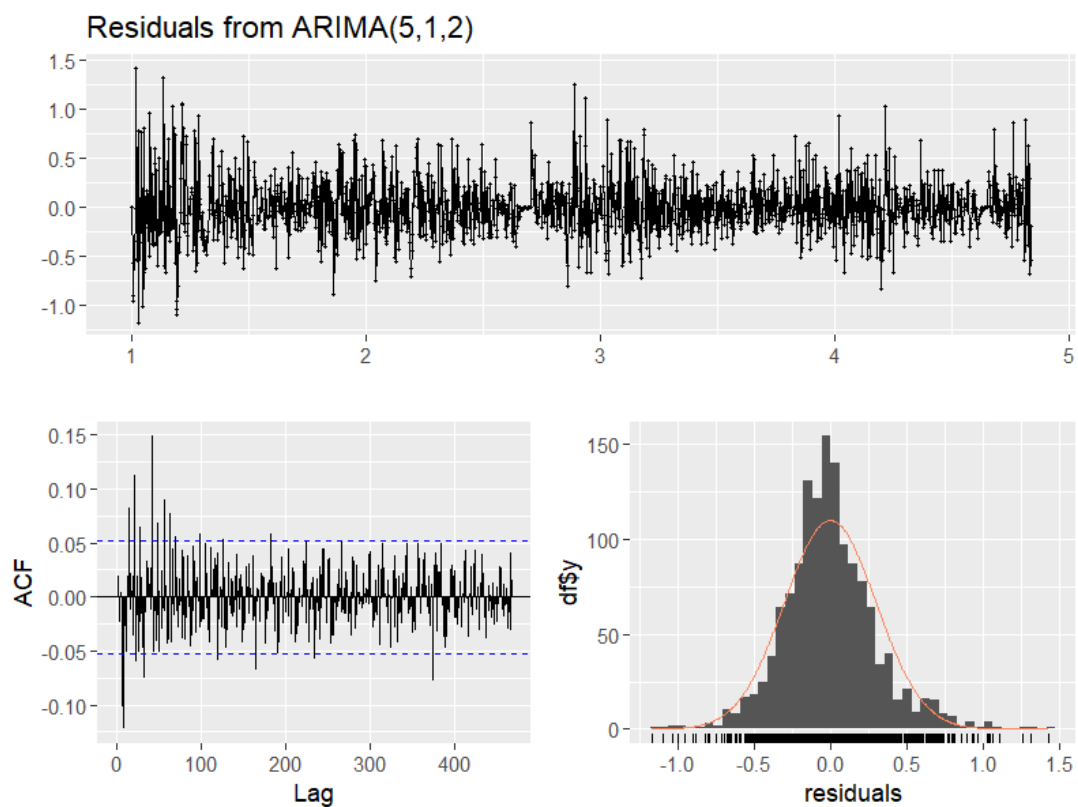
Metodyka: Zastosowano modelowanie autoregresyjne ARIMA. Aby zachować rygor badawczy i uniknąć ewaluacji prognozy bez punktu odniesienia (tzw. „ślepej prognozy”), zastosowano technikę testowania wstecznego (**backtesting**). Szereg czasowy podzielono na dwa podzbiory:

1. **Zbiór uczący (Train):** Wszystkie dane z wyłączeniem ostatnich 30 dni. Na tym zbiorze wytrenowano algorytm `auto.arima()` (minimalizacja kryterium AIC).
2. **Zbiór testowy (Test):** Ostatnie, „ukryte” przed modelem 30 dni obserwacji, służące do weryfikacji wygenerowanej prognozy ex-post.

Identyfikacja modelu i analiza autokorelacji reszt

Algorytm zidentyfikował jako optymalny wariant model **ARIMA(5,1,2)**. Pojawienie się parametru całkowania ($d = 1$) oznacza, że dla tak przyciętego zbioru uczącego algorytm wymusił jednokrotne różnicowanie danych w celu dodatkowej stabilizacji wariancji przed predykcją. Model ponownie odrzucił komponent sezonowy z uwagi na zbyt długi cykl (365 dni).

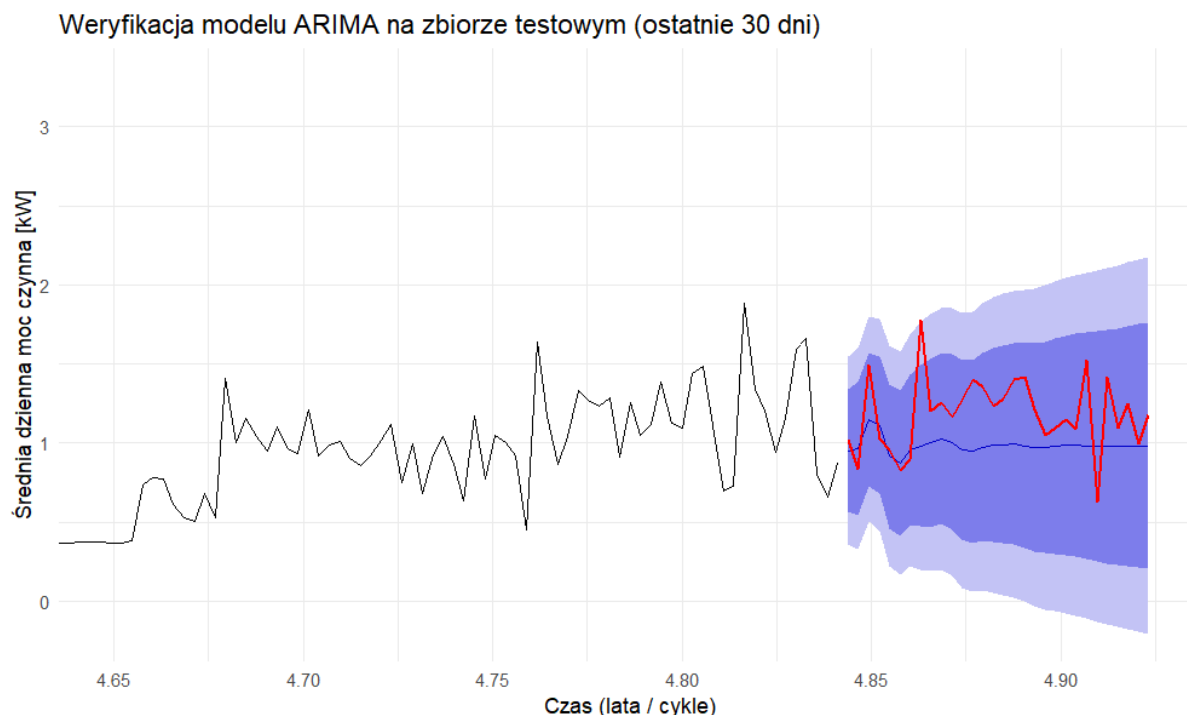
Weryfikację poprawności wyekstrahowania informacji oparto na teście statystycznym Ljung-Boxa. Wartość statystyki $Q^* = 450.15$ oraz p -value na poziomie 1.001×10^{-10} pozwalają na bezwzględne odrzucenie hipotezy zerowej o niezależności reszt. Oznacza to, że reszty nie są białym szumem – wciąż wykazują autokorelację, co wizualnie potwierdzają „wystające” piki na poniższym wykresie ACF. Model nie zdołał wychwycić pełnej struktury danych.



Rysunek 5: Analiza reszt modelu ARIMA(5,1,2) wyestymowanego na zbiorze uczącym.

Weryfikacja predykcji ex-post

Wygenerowano prognozę na 30 dni, a następnie nałożono na nią rzeczywiste dane ze zbioru testowego. Aby wyraźnie zobrazować wariancję, oś czasu przybliżono do ostatnich ok. 100 dni analizowanego okresu.



Rysunek 6: Zderzenie wygenerowanej prognozy ARIMA (niebieska linia i przedziały ufności) z rzeczywistym zużyciem ze zbioru testowego (linia czerwona).

Wykres predykcji dobitnie ukazuje zjawisko **mean reversion** (powrotu do średniej). Prognoza ARIMA(5,1,2) szybko przyjmuje formę wypłaskzonej linii, drastycznie różniąc się z wysoką dynamiką rzeczywistego zapotrzebowania (czerwona linia).

Przyczyny braku dopasowania: Brak komponentu sezonowego w zdefiniowanym modelu spowodował, że w odległym horyzoncie 30 dni algorytm podąża w kierunku uśrednionej wartości historycznej. Złożona, wewnątrz-tygodniowa struktura danych (górk i dołki widoczne na czerwonym przebiegu) jest niemożliwa do przewidzenia przez klasyczny model ARIMA dla danych o rozdzielczości dziennej z wielowarstwową sezonowością.

Wykorzystane polecenia R (part4.R)

```
library(forecast)
library(ggplot2)

cat("--- BUDOWA MODELU ARIMA Z PODZIAŁEM TRAIN/TEST ---\n")

# 1. Podział na zbiór uczący (Train) i testowy (Test)
horyzont <- 30
n <- length(ts_power)
# Train: wszystko oprócz ostatnich 30 dni
train <- subset(ts_power, end = n - horyzont)
# Test: tylko ostatnie 30 dni (ukryte przed modelem)
test <- subset(ts_power, start = n - horyzont + 1)
```

```

# 2. Trening modelu TYLK0 na zbiorze uczącym
model_train <- auto.arima(train, seasonal = TRUE)
print(summary(model_train))

# 3. Analiza reszt (zapis wykresów do pliku)
png("part4-residuals.png", width=800, height=600, res=120)
checkresiduals(model_train)
dev.off()

# Wykonanie i wypisanie formalnego testu Ljung-Boxa
cat("\n--- TEST LJUNG-BOXA ---\n")
print(checkresiduals(model_train, plot=FALSE))

# 4. Wygenerowanie prognozy dla ukrytego okresu (30 dni)
prognoza <- forecast(model_train, h = horyzont)

# 5. Wykres z nałożonymi rzeczywistymi danymi i "zoomem" na końcówkę
png("part4-forecast-overlay.png", width=1000, height=600, res=120)

# Obliczamy punkt startowy do przybliżenia wykresu (ok. 100 dni przed
końcem)
czas_start_zoom <- time(ts_power)[n - 100]
czas_koniec <- time(ts_power)[n]

wykres_overlay <- autoplot(prognoza) +
  autolayer(test, series="Rzeczywiste zużycie", color="red", linewidth=0.8)
+
  coord_cartesian(xlim = c(czas_start_zoom, czas_koniec)) + # Zoom na
kończówkę
  labs(title="Weryfikacja modelu ARIMA na zbiorze testowym (ostatnie 30
dni)",
        x="Czas (lata / cykle)",
        y="Średnia dzienna moc czynna [kW]") +
  theme_minimal() +
  theme(legend.position="bottom")

print(wykres_overlay)
dev.off()

cat("\nZapisano wykres weryfikacyjny do pliku part4-forecast-
overlay.png\n")

```

Część 5. Interpretacja

Wnioski z analizy i ocena wiarygodności

Analiza szeregu czasowego zużycia energii elektrycznej wykazała, że badane zjawisko podlega silnym, cyklicznym odchyleniom addytywnym uwarunkowanym występowaniem pór roku. O ile bazowe zapotrzebowanie jest w miarę stałe (z powolnymi, długoterminowymi fluktuacjami wyłapanymi w dekompozycji STL), o tyle codzienne odchylenia charakteryzują się wysoką dynamiką.

Ryzyko analizy i ocena wiarygodności: Wiarygodność samej diagnozy strukturalnej oraz zaawansowanej dekompozycji szeregu (algorytm STL) oceniam bardzo wysoko. Zasadnicze ryzyko pojawia się na etapie prognozowania. Dzięki rzetelnemu zastosowaniu metodologii Train-Test Split wykazano ponad wszelką wątpliwość, że wygenerowana prognoza z modelu autoregresyjnego ARIMA nie nadaje się do wdrożenia. Opieranie decyzji infrastrukturalnych (np. kontraktowania energii do sieci typu smart-grid) na wypłaszczonym modelu doprowadziłoby do powstawania poważnych niedoborów lub nadwyżek mocy w krótkich odstępach czasu.

Lessons learned

Przeprowadzenie pełnego cyklu analitycznego pozwoliło na zidentyfikowanie kluczowych pułapek oraz dobrych praktyk badawczych:

- **Co poszło dobrze:** Zastosowanie techniki **Backtestingu** (podział na zbiór uczący i testowy) było najlepszą decyzją metodyczną w tym projekcie. Uchroniło to analizę przed błędem „nadmiernego optymizmu” względem w pełni zautomatyzowanego algorytmu `auto.arima()` i pozwoliło rzetelnie ocenić model. Dodatkowo, elastyczne zastąpienie niedoskonałej dekompozycji klasycznej algorytmem STL dowiodło, że weryfikacja wizualna wyników z wbudowanych funkcji jest niezbędna.
- **Co można zrobić inaczej (Rekomendacje):** Analizowany szereg wykazuje tzw. „podwójną sezonowość” (wahania tygodniowe nałożone na roczne). Ponieważ klasyczne modele ARIMA z reguły odrzucają sezonowość o okresie przekraczającym 350 obserwacji, w przyszłości należy zrezygnować z tej rodziny algorytmów dla zagregowanych danych dziennych o rocznym cyklu. Konieczne jest użycie specjalistycznych architektur, takich jak modele **TBATS** (Trigonometric seasonality, Box-Cox transformation, ARMA errors, Trend and Seasonal components) lub opartych na głębokich sieciach neuronowych (**LSTM**).